МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Пермский государственный национальный исследовательский университет

ОТЧЁТ

по лабораторной работе № 2

«Многопоточное умножение матриц»

Выполнила: Витухина Анастасия
Группа: ИТ-7-8

Пермь — 2026

1. Цель работы

Изучить принципы многопоточного умножения матриц, распределение вычислений между потоками и синхронизацию общего счётчика. Дополнительно реализовать генерацию матриц отдельным процессом с передачей данных через IPC, компактное хранение нижнетреугольной матрицы и сравнение статического и динамического распределения строк.

2. Среда выполнения

Измерения необходимо проводить на основной операционной системе компьютера, не в виртуальной машине. Значения ниже заполняются по фактическим характеристикам компьютера, на котором выполнены замеры; характеристики другого компьютера использовать нельзя.

Параметр	**Фактическое значение**
Модель процессора	Intel Core i5-13420H
Физические ядра	8
Логические процессоры	12
Типы ядер (если гибридный CPU)	4 производительных P-core + 4 энергоэффективных E-core
Кэш L3	12 МБ
Оперативная память	15,7 ГБ (около 16 ГБ)
Операционная система и версия	Windows 11 Домашняя для одного языка, 10.0.26200
Версия Java / JDK	Eclipse Temurin OpenJDK 25.0.4.1 LTS
Среда разработки	Visual Studio Code
Место проведения замеров	Windows 11 на основном компьютере; виртуальная машина не использовалась.
Компиляция и запуск	javac -encoding UTF-8; запуск в JVM Eclipse Temurin. Перед каждым измерением выполняется прогревочный запуск; отдельные флаги оптимизации javac не задавались.

3. Описание реализации

3.1. Обычное умножение матриц

Матрица A имеет размер n × m, матрица B — m × k, результирующая матрица C — n × k. Элемент результата вычисляется вручную тремя вложенными циклами:

$C[i][j] = \Sigma(A[i][p] \cdot B[p][j]), p = 0, \ldots, m-1.$

Матрицы хранятся в двумерных массивах long[][]. Исходные элементы генерируются случайно в диапазоне от 0 до 9. Используется порядок циклов i → p → j. Полная матрица C собирается в памяти программы, а после вычисления выводится контрольная сумма.

3.2. Распределение строк и счётчик прогресса

Рабочие потоки создаются вручную с помощью класса Thread. Строки C делятся на непрерывные блоки. Если n не делится на T, первые потоки получают по одной дополнительной строке; если T больше n, часть потоков получает пустой блок. Главный поток ожидает завершения рабочих потоков методом join().

После завершения каждой строки поток увеличивает общий счётчик completed типа AtomicInteger. Атомарность предотвращает потерю обновлений, когда несколько потоков одновременно изменяют счётчик. Блокировка матрицы C не нужна, поскольку потоки записывают в непересекающиеся строки.

3.3. Генерация матриц в отдельном процессе и IPC

Для режима ipc родительская программа запускает отдельную JVM через ProcessBuilder. Дочерний процесс GeneratorProcess генерирует матрицы A и B и передаёт их родительскому процессу через стандартный поток вывода. Передача выполняется в бинарном формате с помощью DataOutputStream и DataInputStream: сначала записываются размеры, затем элементы типа long. Служебные сообщения дочернего процесса направляются в stderr, чтобы не повредить бинарный поток.

Данные проходят через буфер анонимного канала, обслуживаемого ОС: дочерний процесс записывает байты, а родительский процесс считывает их. Поэтому при передаче между процессами данные копируются через системный буфер. Если родитель читает медленнее, буфер заполняется и запись дочернего процесса блокируется до освобождения места. В начале потока передаются размеры матриц и длительность генерации; получатель затем считывает строго известное количество long-элементов. Для одного производителя и одного потребителя

дополнительный mutex не требуется: порядок байтов сохраняется, а блокирующие операции канала обеспечивают согласование скорости. Завершение дочернего процесса дополнительно проверяется по коду выхода.

Время запуска процесса, генерации матриц в дочернем процессе, чтения полезных данных и полный этап получения матриц измеряются отдельно. Полное время IPC включает запуск дочерней JVM, получение заголовка и данных, а также ожидание завершения процесса. Для каждого размера выполнено пять сравнительных замеров при T = 12; итоговые значения рассчитаны как усечённое среднее после исключения минимума и максимума.

3.4. Сравнение способов межпроцессного взаимодействия

Способ IPC	Скорость для больших данных	Простота	Синхронизация	Между ПК	Переносимость
Анонимный канал (pipe), выбран	Высокая для последовательного обмена на одном компьютере; данные проходят через буфер ОС.	Простой для родителя и дочернего процесса.	Блокирующее чтение/запись обеспечивает потоковый обмен; отдельный mutex не нужен для одного отправителя.	Нет, локальный IPC.	Высокая; конкретный API зависит от ОС.
Файл	Ниже для больших объёмов из-за операций с файловой системой и накопителем.	Очень простой и легко отлаживать.	Нужны соглашение о готовности файла, блокировка или атомарное переименование.	Да, через общую сетевую папку, если она доступна.	Очень высокая.
Сетевой сокет (TCP)	Высокая, но есть накладные расходы протокола и сети.	Средняя: нужно описать протокол и обработку ошибок.	Нужно определить границы сообщений и порядок обмена.	Да.	Высокая; TCP поддерживается основными ОС.
Разделяемая память / отображаемый файл	Очень высокая на одном компьютере для больших объёмов: можно избежать повторной передачи через канал.	Сложнее в настройке и отладке.	Нужна явная синхронизация доступа и сигнализация готовности.	Нет, общая память локальна для одного компьютера.	Средняя; API и детали зависят от ОС.

Способ IPC	Скорость для больших данных	Простота	Синхронизация	Между ПК	Переносимость
Очередь сообщений	Средняя; удобна для небольших сообщений, крупные массивы требуют разбиения.	Средняя.	Очередь упорядочивает сообщения, но прикладной протокол всё равно нужен.	Обычно нет; зависит от реализации брокера/системы.	Средняя или низкая; системные интерфейсы различаются.

Выбран канал, поскольку генератор запускается родительской программой, данные передаются последовательно, а бинарный формат позволяет передать размеры и элементы матриц без текстового преобразования. Для этой задачи не требуется сетевое взаимодействие или постоянное хранение исходных данных.

3.5. Нижнетреугольная матрица

Матрица A является нижнетреугольной: все элементы выше главной диагонали равны нулю. Эти нулевые элементы не хранятся. Строка с индексом i содержит только i + 1 значимых элементов. Всего хранится n(n + 1)/2 элементов вместо n^2. Матрица B остаётся обычной квадратной матрицей.

Элемент результата вычисляется по формуле $C[i][j] = \Sigma(A[i][p] \cdot B[p][j])$, где p принимает значения от 0 до i. В строках нижнетреугольной матрицы разное число элементов, поэтому объём вычислений по строкам неодинаков.

3.6. Статическое и динамическое распределение

При статическом распределении каждому потоку заранее назначается непрерывный блок строк. При динамическом распределении поток получает следующую свободную строку с помощью AtomicInteger.getAndIncrement(). Статический способ имеет небольшие накладные расходы на выдачу работы, динамический может лучше балансировать неравномерную нагрузку. Какой способ быстрее, определяется только по результатам измерений.

3.7. Проверка корректности

После параллельного вычисления программа сравнивает результат с независимым последовательным умножением поэлементно. Проверка реализована для режимов multiply, ipc,

triangle static и triangle dynamic. Контрольная сумма используется как дополнительная характеристика, но не заменяет поэлементное сравнение.

3.8. Ключевые фрагменты реализации

Распределение строк и обновление общего счётчика выполняются вручную. AtomicInteger обеспечивает атомарное увеличение счётчика, а потоки записывают результат в непересекающиеся строки матрицы C.

```
workers[t] = new Thread(() -> {
    for (int i = from; i < to; i++) {
        for (int p = 0; p < m; p++) {
            long value = a[i][p];
            for (int j = 0; j < k; j++)
                c[i][j] += value * b[p][j];
        }
        completed.incrementAndGet();
    }
});
```

Для динамического распределения нижнетреугольной матрицы следующая строка выдаётся потоку через общий атомарный индекс:

```
int row;
while ((row = nextRow.getAndIncrement()) < n) {
    calculateRow(a, b, c, row, k);
    completed.incrementAndGet();
}
```

Запуск дочернего процесса и передача бинарных данных реализованы через ProcessBuilder и потоки DataOutputStream / DataInputStream. Получатель сначала читает размеры, затем известное количество элементов типа long.

Сборка выполнялась командой javac -encoding UTF-8. Измерения выполнялись на JVM Eclipse Temurin OpenJDK 25.0.4.1 LTS; перед каждым учитываемым запуском выполнялся неучитываемый прогревочный запуск.

Проверены небольшие матрицы: multiply для A 7 × 5 и B 5 × 4; IPC для тех же размеров; triangle static и dynamic для матрицы 7 × 7. При контрольных запусках сравнение с последовательным алгоритмом завершилось успешно.

4. Методика измерений

Для измерения используется System.nanoTime(), монотонный таймер высокого разрешения. Генерация исходных матриц выполняется до измерения. В замер входят создание рабочих потоков, вычисление и ожидание завершения. Подсчёт контрольной суммы и запись строки CSV выполняются после остановки таймера. Перед каждым измерением выполняется один прогревочный запуск, не включаемый в результат.

4.1. Обычное умножение

Полная серия: квадратные матрицы 500 × 500, 1500 × 1500 и 3000 × 3000. На компьютере JVM обнаружила L = 12 логических процессоров. Проверялись T = 1…24, а также 48 (4L) и 96 (8L), то есть 26 значений числа потоков. Для каждой пары «размер — число потоков» выполнено пять измерений.

Для каждой комбинации сохраняются пять исходных времён. При расчёте среднего минимальное и максимальное значения отбрасываются, а три оставшихся усредняются. Ускорение $S(T) = t(1)/t(T)$, эффективность $E(T) = S(T)/T$.

4.2. Нижнетреугольная матрица

Для тех же размеров и того же набора значений T выполняется сравнение static и dynamic. Пять повторов для обоих способов проводятся на одинаковых исходных матрицах. Результаты записываются в triangle_benchmark.csv. При анализе применяется усечённое среднее: удаляются минимальное и максимальное значения, три оставшихся усредняются.

4.3. Сравнение IPC и умножения

Для размеров 500 × 500, 1500 × 1500 и 3000 × 3000 запускается отдельный процесс-генератор. В IPC-серии использовалось T = 12. Отдельно фиксировались запуск дочерней JVM, генерация, чтение полезных данных по каналу, полное время получения матриц и время последующего умножения. Для каждого размера выполнено пять измерений; из каждого набора исключались минимум и максимум, после чего усреднялись три оставшихся значения.

5. Результаты измерений

Для обычного умножения и нижнетреугольной матрицы приведены полные серии измерений: по пять повторов для каждой комбинации. Режимы --quick в итоговые результаты не

включались. Для IPC также выполнено по пять сравнительных замеров при T = 12; в таблице и на графике представлены усечённые средние.

5.1. Обычное умножение

Матрицы 500 × 500

Потоки T	Усечённое среднее, с	Ускорение S(T)	Эффективность E(T)
1	0.028148	1.000	1.000
2	0.015520	1.814	0.907
3	0.009980	2.821	0.940
4	0.007625	3.691	0.923
5	0.008418	3.344	0.669
6	0.007166	3.928	0.655
7	0.006679	4.214	0.602
8	0.006308	4.462	0.558
9	0.006640	4.239	0.471
10	0.006852	4.108	0.411
11	0.005773	4.876	0.443
12	0.005679	4.957	0.413
13	0.005654	4.979	0.383
14	0.005850	4.812	0.344
15	0.005920	4.755	0.317
16	0.005697	4.941	0.309
17	0.005970	4.715	0.277
18	0.006981	4.032	0.224
19	0.005845	4.816	0.253
20	0.006535	4.307	0.215
21	0.006577	4.280	0.204
22	0.005796	4.856	0.221
23	0.006123	4.597	0.200
24	0.005868	4.797	0.200
48	0.006736	4.179	0.087
96	0.008038	3.502	0.036

Матрицы 1500 × 1500

Потоки T	Усечённое среднее, с	Ускорение S(T)	Эффективность E(T)
1	1.499051	1.000	1.000
2	0.877783	1.708	0.854
3	0.586157	2.557	0.852
4	0.416312	3.601	0.900
5	0.427957	3.503	0.701
6	0.376247	3.984	0.664
7	0.289735	5.174	0.739
8	0.275528	5.441	0.680
9	0.274716	5.457	0.606
10	0.241197	6.215	0.622
11	0.264816	5.661	0.515
12	0.253844	5.905	0.492
13	0.252360	5.940	0.457
14	0.235298	6.371	0.455
15	0.242322	6.186	0.412
16	0.234473	6.393	0.400
17	0.248472	6.033	0.355
18	0.259128	5.785	0.321
19	0.264812	5.661	0.298
20	0.257298	5.826	0.291
21	0.259134	5.785	0.275
22	0.242690	6.177	0.281
23	0.257174	5.829	0.253
24	0.269342	5.566	0.232
48	0.270578	5.540	0.115
96	0.246913	6.071	0.063

Матрицы 3000 × 3000

Потоки T	Усечённое среднее, с	Ускорение S(T)	Эффективность E(T)
1	14.841487	1.000	1.000
2	8.038217	1.846	0.923

Потоки T	Усечённое среднее, с	Ускорение S(T)	Эффективность E(T)
3	6.164989	2.407	0.802
4	5.123015	2.897	0.724
5	4.111604	3.610	0.722
6	3.692989	4.019	0.670
7	3.582938	4.142	0.592
8	3.590776	4.133	0.517
9	3.901265	3.804	0.423
10	4.076391	3.641	0.364
11	3.728714	3.980	0.362
12	3.811020	3.894	0.325
13	3.606728	4.115	0.317
14	3.748589	3.959	0.283
15	3.642909	4.074	0.272
16	3.481076	4.263	0.266
17	3.412679	4.349	0.256
18	3.419032	4.341	0.241
19	3.471307	4.275	0.225
20	3.468964	4.278	0.214
21	3.337746	4.447	0.212
22	3.578714	4.147	0.189
23	3.344838	4.437	0.193
24	3.537451	4.196	0.175
48	3.176553	4.672	0.097
96	3.035466	4.889	0.051

5.2. Нижнетреугольная матрица: static и dynamic

Размер	T	Среднее static, с	Среднее dynamic, с	Быстрее
500 × 500	1	0.019620	0.017563	dynamic
500 × 500	2	0.012510	0.009192	dynamic
500 × 500	3	0.008252	0.007823	dynamic
500 × 500	4	0.006892	0.006899	static
500 × 500	5	0.005135	0.005186	static
500 × 500	6	0.004516	0.004724	static
500 × 500	7	0.004442	0.004563	static
500 × 500	8	0.004790	0.004667	dynamic
500 × 500	9	0.004072	0.004218	static
500 × 500	10	0.004538	0.004304	dynamic
500 × 500	11	0.004666	0.005100	static
500 × 500	12	0.004417	0.004784	static
500 × 500	13	0.004050	0.004489	static
500 × 500	14	0.004078	0.004515	static
500 × 500	15	0.004459	0.005050	static
500 × 500	16	0.003992	0.005314	static
500 × 500	17	0.004164	0.004903	static
500 × 500	18	0.004248	0.004817	static
500 × 500	19	0.003843	0.004759	static
500 × 500	20	0.003905	0.004612	static
500 × 500	21	0.004156	0.004675	static
500 × 500	22	0.003777	0.004742	static
500 × 500	23	0.004090	0.004800	static
500 × 500	24	0.003871	0.005040	static
500 × 500	48	0.004722	0.006390	static
500 × 500	96	0.006220	0.009189	static
1500 × 1500	1	0.792252	0.999437	static
1500 × 1500	2	0.705402	0.417883	dynamic
1500 × 1500	3	0.398071	0.265205	dynamic
1500 × 1500	4	0.319568	0.229384	dynamic
1500 × 1500	5	0.252472	0.171673	dynamic
1500 × 1500	6	0.237889	0.147829	dynamic
1500 × 1500	7	0.197274	0.155889	dynamic
1500 × 1500	8	0.180525	0.152135	dynamic
1500 × 1500	9	0.167347	0.144537	dynamic
1500 × 1500	10	0.167064	0.144534	dynamic
1500 × 1500	11	0.173857	0.134873	dynamic
1500 × 1500	12	0.158866	0.142325	dynamic
1500 × 1500	13	0.161255	0.141256	dynamic

Размер	T	Среднее static, с	Среднее dynamic, с	Быстрее
1500 × 1500	14	0.144547	0.136043	dynamic
1500 × 1500	15	0.149011	0.129461	dynamic
1500 × 1500	16	0.143113	0.131727	dynamic
1500 × 1500	17	0.162041	0.133907	dynamic
1500 × 1500	18	0.137206	0.122633	dynamic
1500 × 1500	19	0.134837	0.127802	dynamic
1500 × 1500	20	0.126329	0.123775	dynamic
1500 × 1500	21	0.131992	0.121177	dynamic
1500 × 1500	22	0.132497	0.125545	dynamic
1500 × 1500	23	0.139633	0.118765	dynamic
1500 × 1500	24	0.122597	0.115022	dynamic
1500 × 1500	48	0.115661	0.128058	static
1500 × 1500	96	0.115590	0.128501	static
3000 × 3000	1	6.815266	7.519208	static
3000 × 3000	2	5.643154	4.129417	dynamic
3000 × 3000	3	4.476700	3.030944	dynamic
3000 × 3000	4	3.606445	2.401592	dynamic
3000 × 3000	5	3.132719	1.994216	dynamic
3000 × 3000	6	2.797449	1.772173	dynamic
3000 × 3000	7	2.594967	1.591944	dynamic
3000 × 3000	8	2.395086	1.473104	dynamic
3000 × 3000	9	2.289542	1.359160	dynamic
3000 × 3000	10	2.155780	1.326397	dynamic
3000 × 3000	11	2.063531	1.257011	dynamic
3000 × 3000	12	3.100829	2.663631	dynamic
3000 × 3000	13	3.811809	2.840849	dynamic
3000 × 3000	14	3.227236	2.829106	dynamic
3000 × 3000	15	3.212546	2.759026	dynamic
3000 × 3000	16	2.866907	1.955691	dynamic
3000 × 3000	17	2.257550	1.746382	dynamic
3000 × 3000	18	2.352776	1.817714	dynamic
3000 × 3000	19	2.343449	1.784269	dynamic
3000 × 3000	20	2.183360	1.777427	dynamic
3000 × 3000	21	2.151957	1.801292	dynamic
3000 × 3000	22	3.039582	2.799383	dynamic
3000 × 3000	23	2.633188	2.565996	dynamic
3000 × 3000	24	2.349362	1.804442	dynamic
3000 × 3000	48	2.008019	1.788718	dynamic
3000 × 3000	96	1.813675	1.774674	dynamic

5.3. Сравнение IPC и умножения

Размер матриц	T	Запуск, с	Генерация, с	Передача, с	Весь этап IPC, с	Умножение, с
500 × 500	12	0.035534	0.015951	0.893970	1.042524	0.019928
1500	12	0.008267	0.097173	7.123968	7.347469	0.428065

× 1500						
3000 × 3000	12	0.006645	0.386035	28.324977	28.895621	5.292388

5.4. Графики

Представлены обязательные графики обычного умножения для всех размеров, сравнение static/dynamic для каждого размера нижнетреугольной матрицы и сравнение времени передачи данных по IPC со временем умножения.

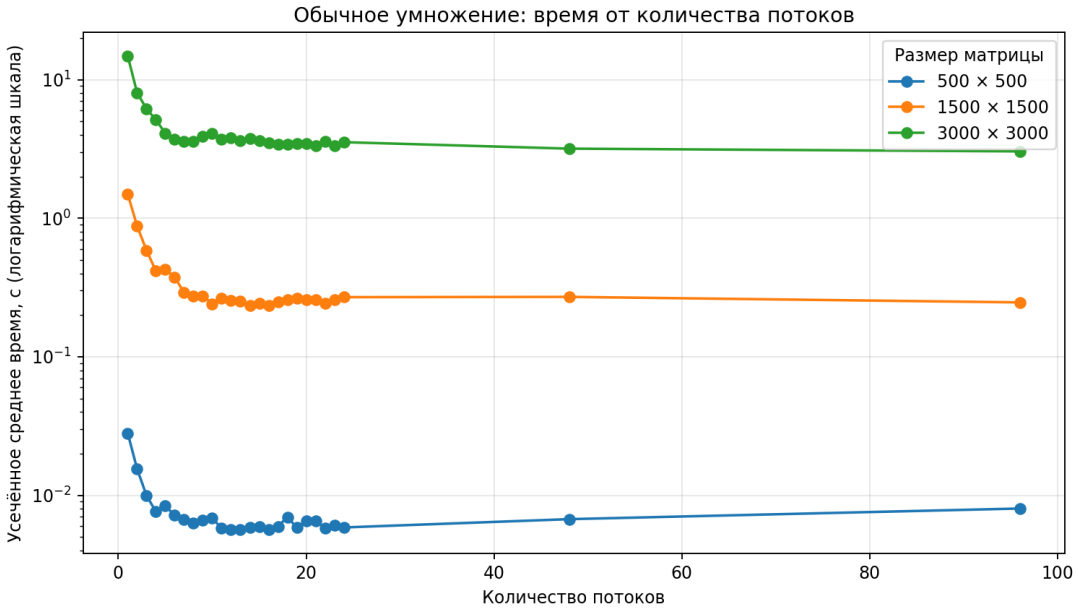


Рисунок 1 — Время умножения для всех трёх размеров матриц.

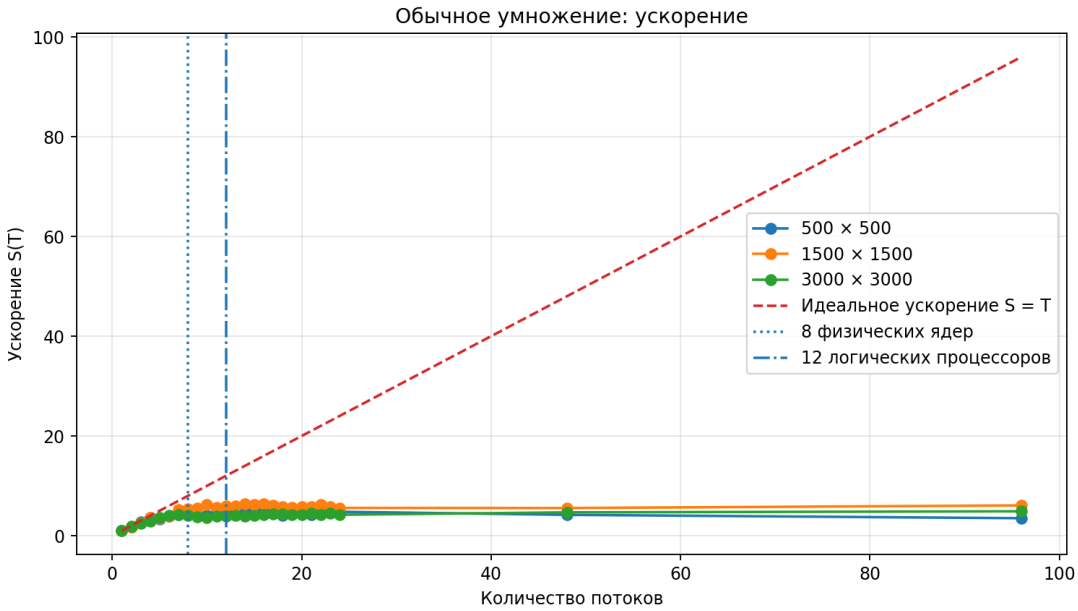


Рисунок 2 — Ускорение для всех размеров, идеальная линия S = T и отметки ядер процессора.

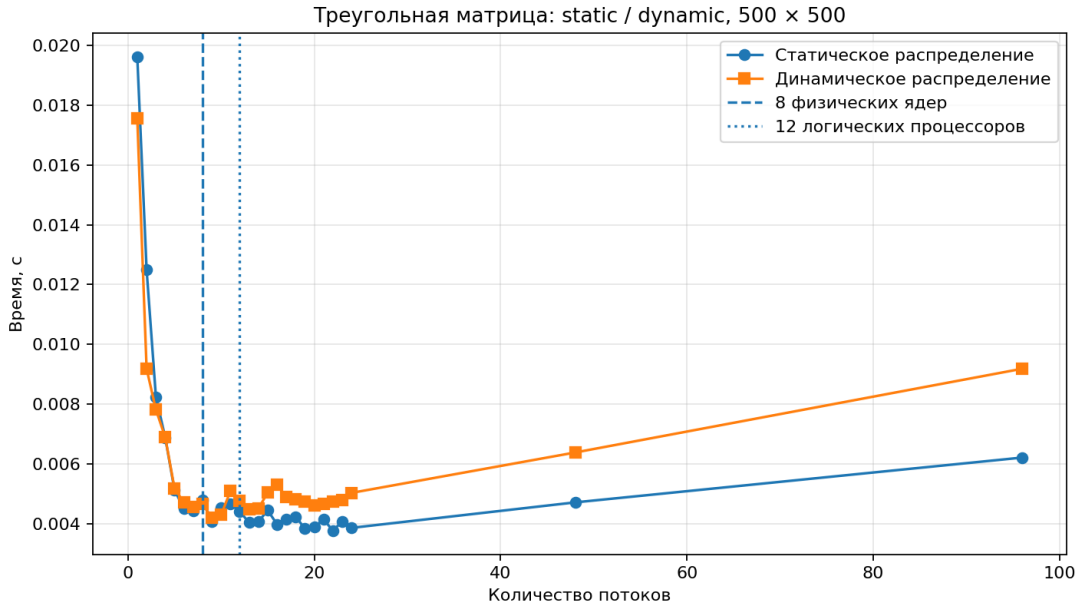


Рисунок 3 — Static и dynamic для нижнетреугольной матрицы 500 × 500.

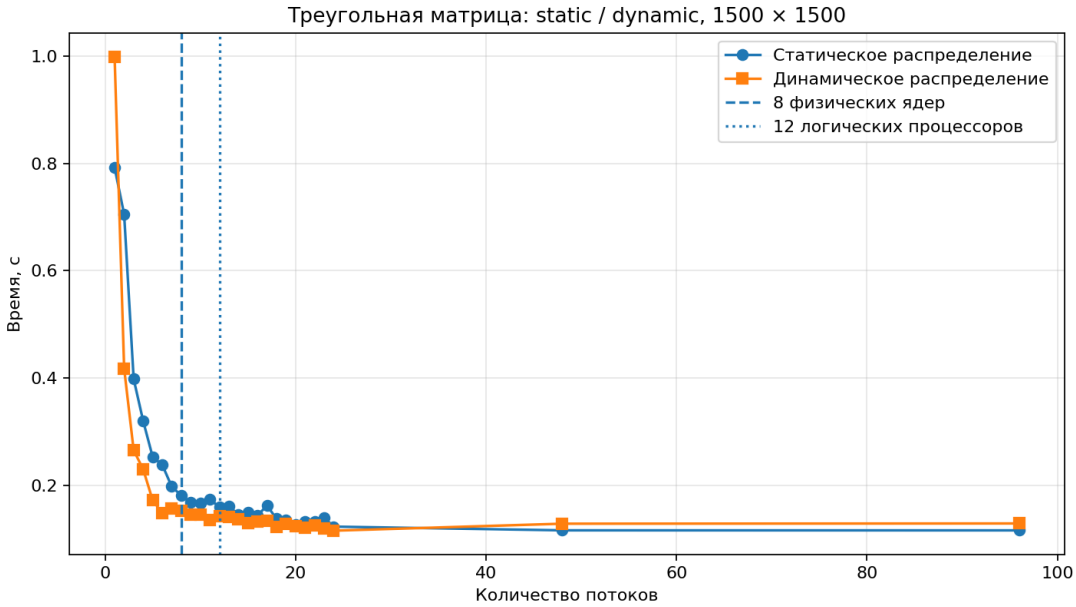


Рисунок 4 — Static и dynamic для нижнетреугольной матрицы 1500 × 1500.

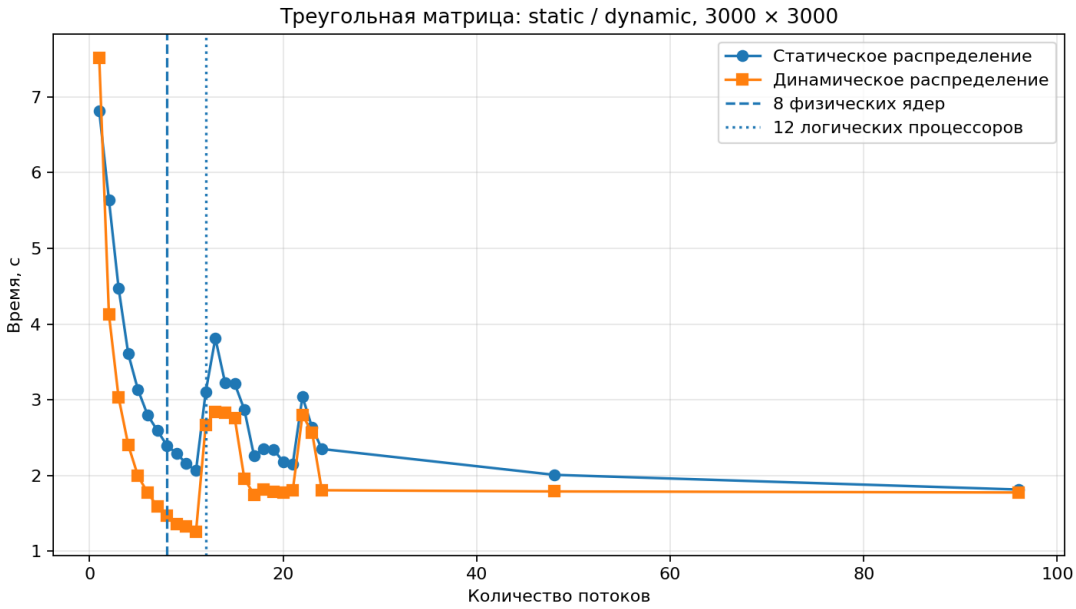


Рисунок 5 — Static и dynamic для нижнетреугольной матрицы 3000 × 3000.

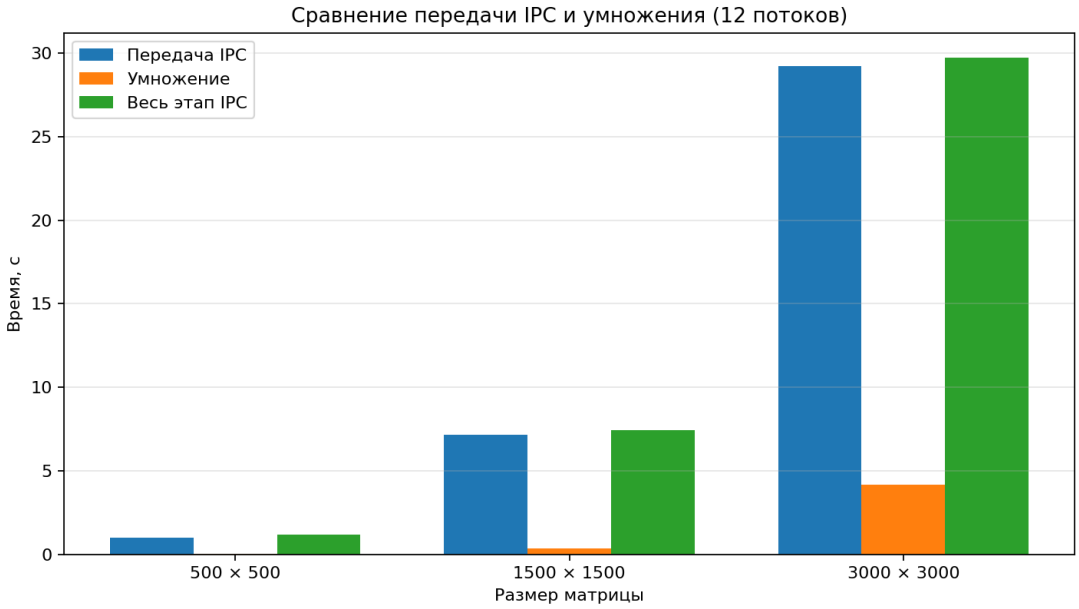


Рисунок 6 — Сравнение передачи данных по IPC и времени умножения.

6. Анализ результатов

Какое число потоков оказалось оптимальным.

Оптимум среди проверенных значений зависит от размера матрицы. Для 500 × 500 наименьшее усечённое среднее получено при T = 13: 0,005654 с (ускорение 4,98 раза относительно T = 1); при T = 12 результат почти такой же — 0,005679 с. Для 1500 × 1500 лучший результат получен при T = 16: 0,234473 с (ускорение около 6,40 раза). Для 3000 × 3000 минимальное время среди проверенных конфигураций получено при T = 96: 3,035466 с (ускорение около 4,89 раза). Это экспериментальный минимум в проверенном наборе, а не универсальный оптимум для любого компьютера.

До какого числа потоков ускорение.

На первых четырёх потоках масштабирование для матриц 500 × 500 и 1500 × 1500 близко к линейному: ускорение составляет примерно 3,69 и 3,60 раза соответственно. Для 3000 × 3000 ускорение при T = 4 ниже — около 2,90 раза. При переходе от 8 физических ядер к 12 логическим результат зависит от размера: матрицы 500 и 1500 ускоряются ещё примерно на 10% и 8%, а для 3000 время при T = 12 немного увеличивается по сравнению с T = 8.

Что произошло при количестве потоков.

После 12 потоков дополнительное ускорение не гарантировано. Для 500 × 500 лучший результат получен при T = 13, после чего время в целом растёт; для 1500 × 1500 минимум получен при T = 16, а 48 и 96 потоков уже не улучшают его. Для 3000 × 3000 большая вычислитель х. ная задача продолжает выигрывать от большего числа потоков и лучший измеренный результат получен при T = 96, хотя прирост относительно 48 потоков уже небольшой. Избыточные потоки добавляют затраты на создание, планирование и переключение контекста, а также конкурируют за ресурсы процессора.

Как размер матрицы повлиял.

Размер заметно меняет масштабирование. При хранении трёх квадратных матриц A, B и C типа long их суммарный объём составляет примерно 5,72 МиБ для 500 × 500, 51,50 МиБ для 1500 × 1500 и 205,99 МиБ для 3000 × 3000. При кэше L3 объёмом 12 МиБ набор для 500 × 500 теоретически может поместиться в L3, тогда как наборы для 1500 и 3000 — нет. Поэтому малый размер быстрее достигает плато, а крупный сохраняет пользу от большего числа потоков дольше.

Что сильнее ограничивало ускорение.

По одним только замерам нельзя строго отделить ограничение вычислительных ядер от ограничения пропускной способности памяти: отдельный тест памяти не проводился. Однако объём рабочих данных для 1500 × 1500 и 3000 × 3000 существенно превышает L3, а ускорение заметно ниже идеального. Следовательно, на результат одновременно влияют доступность вычислительных ресурсов, иерархия кэшей и обмен с оперативной памятью. Порядок циклов i → p → j обеспечивает последовательный проход по строкам B и C во внутреннем цикле j, что улучшает локальность доступа.

Для нижнетреугольной матрицы.

Динамический способ оказался быстрее в 53 из 78 сравнений. Его преимущество особенно заметно на крупных матрицах: для 3000 × 3000 при T = 8 время dynamic составило 1,473104 с против 2,395086 с у static (примерно на 38,5% меньше); при T = 12 — 2,663631 с против 3,100829 с (на 14,1% меньше). Для 500 × 500 динамический способ выиграл лишь в 5 из 26 конфигураций. Причина — неодинаковая трудоёмкость строк: поздние строки содержат больше элементов и требуют больше операций. Динамическая выдача строк через AtomicInteger

лучше балансирует нагрузку, но атомарное получение каждой строки создаёт дополнительные накладные расходы, заметные на малых матрица

Как соотносятся время IPC.

При T = 12 усечённое среднее время передачи по каналу составило 0,893970 с для 500 × 500, 7,123968 с для 1500 × 1500 и 28,324977 с для 3000 × 3000. Умножение заняло соответственно 0,019928 с, 0,428065 с и 5,292388 с. Полный этап IPC составил 1,042524 с, 7,347470 с и 28,895621 с — примерно в 52,3; 17,2 и 5,5 раза больше времени умножения. Доля собственно передачи в полном этапе IPC составила около 85,8%, 96,9% и 98,0%. Отношение IPC/умножение уменьшается с ростом размера, поскольку вычисление растёт быстрее, но передача во всех трёх случаях остаётся дороже умножения. Значения рассчитаны по пяти измерениям с исключением минимума и максимума.

7. Вывод

В лабораторной работе реализованы многопоточное умножение обычных матриц, вывод прогресса, синхронизация общего счётчика и проверка результата последовательным алгоритмом. Дополнительно реализованы генерация матриц отдельным процессом с передачей через анонимный канал и компактное хранение нижнетреугольной матрицы. Для обычного умножения и режимов static/dynamic выполнено по пять измерений для трёх размеров матриц и 26 значений числа потоков. Для матрицы 3000 × 3000 усечённое среднее при одном потоке составило 14,841487 с, а лучший результат среди проверенных конфигураций — 3,035466 с при 96 потоках. Для размеров 500 и 1500 лучшие результаты получены при 13 и 16 потоках соответственно. Динамическое распределение оказалось быстрее статического в 53 из 78 сравнений, особенно на крупных матрицах. Для IPC также выполнено по пять измерений: усечённое среднее полного этапа получения матриц для размера 3000 × 3000 составило 28,895621 с, что значительно больше времени умножения — 5,292388 с. Эксперименты показали, что оптимальное число потоков зависит от размера задачи, а накладные расходы потоков, кэш и передача данных существенно влияют на итоговую производительность.

8. Приложение. Сборка и запуск

Сборка в Windows PowerShell из корня проекта:

```
New-Item -ItemType Directory -Force out
javac -encoding UTF-8 -d out (Get-ChildItem src/lab2/*.java).FullName
```

Контрольные режимы:

```
java -cp out lab2.Main multiply 7 5 4 3
java -cp out lab2.Main ipc 7 5 4 3
java -cp out lab2.Main triangle 7 3 static
java -cp out lab2.Main triangle 7 3 dynamic
```

Полные эксперименты:

java -cp out lab2.Benchmark
java -cp out lab2.Benchmark triangle
java -cp out lab2.Benchmark ipc 12

Построение графиков по CSV (после установки matplotlib):
py -m pip install matplotlib
py plot_results.py --physical-cores 8 --logical-processors 12

В приложение к отчёту приложить исходный код, README.md, полные CSV-файлы и построенные графики. Итоговые таблицы и выводы заполняются только после завершения полных экспериментов.